SLOVENSKÁ TECHNICKÁ
UNIVERZITA V BRATISLAVE
FAKULTA INFORMATIKY
A INFORMAČNÝCH TECHNOLÓGIÍ

Infraštruktúra verejného kľúča

Meno: Juraj Černička

ID: 127128

Rok: 2024/2025

Cvičenie: doc. Ing. Peter Trúchly, PhD., Streda 8.00

Obsah

1. Princíp asymetrickej kryptografie....3
1.1. Základný princíp....3
1.2. Digitálne pečiatky....5
1.3. RSA....6
1.4. Kvantové počítanie....9
1.5. Digitálne certifikáty....10
2. Praktická časť....15
2.1. Triedy....16
2.2. Reprezentácia kľúčov....17
2.3. JSON requesty....18
2.4. Konzolové príkazy....20
2.5. Detaily šifrovania....23
2.6. Zmeny od originálneho plánu....23
3. Záver....24
3.1. Zdroje....24

1. Princíp asymetrickej kryptografie

1.1 Základný princíp

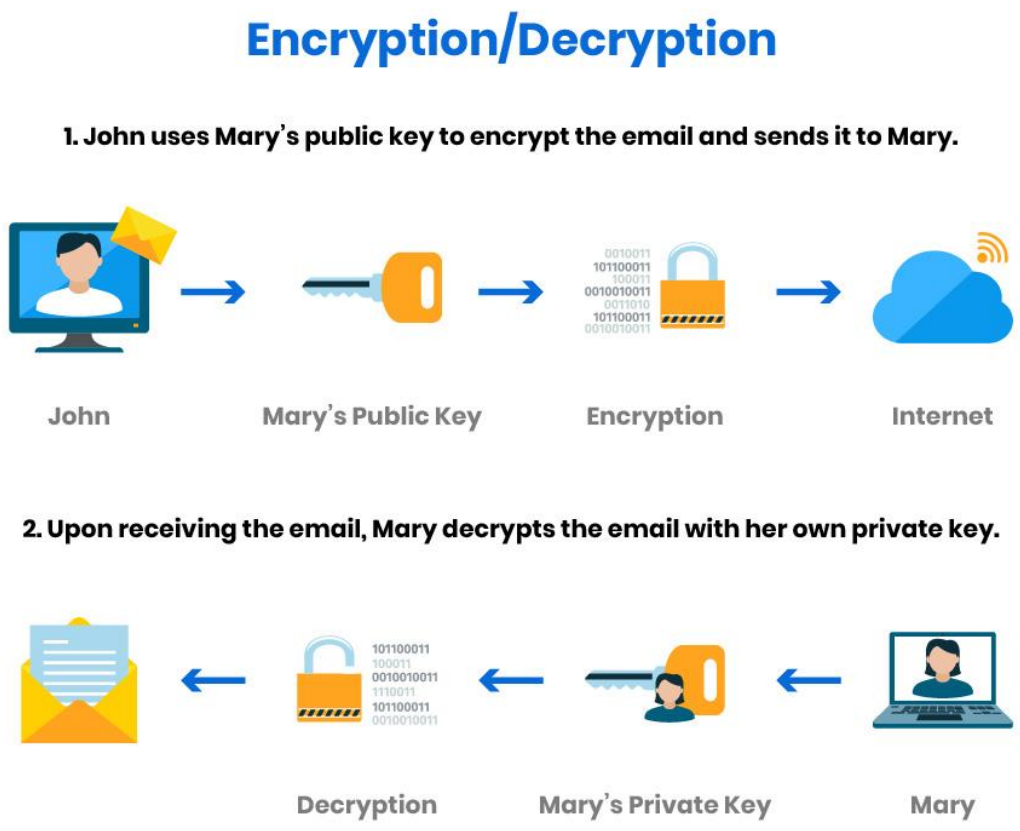
Asymetrická kryptografia je spôsob šifrovania a dešifrovania dát, ktorý používa dvojicu kľúčov: verejný kľúč a súkromný kľúč. Tieto kľúče predstavujú základný pilier asymetrického šifrovania.

Každá entita, ktorá chce úspešne komunikovať pomocou asymetrického šifrovania musí mať unikátny verejný aj súkromný kľúč. Verejný kľúč je podľa názvu verejne dostupný pre kohokoľvek, kto chce s danou entitou komunikovať. Ku súkromnému kľúču má naopak prístup jedine entita, ktorej daný kľúč patrí. Na tajnosti súkromného kľúča je založený celý systém kryptografie.

Ak sa niekto rozhodne poslať entite šifrované správy, zoberie jej verejný kľúč, ku ktorému by mal mať voľný prístup, a pomocou verejného kľúča správu zašifruje. Systém šifrovania je vytvorený tak, aby sa správa zašifrovaná verejným kľúčom dala odšifrovať jedine súkromným kľúčom, ktorý k nemu patrí. Každá dvojica kľúčov je vygenerovaná spoločne, kde sa verejný kľúč odvodzuje od súkromného.

Verejný kľúč je vlastnou podmnožinou súkromného. Nachádzajú sa v ňom jedine informácie potrebné na zašifrovanie správy, zatiaľ čo súkromný kľúč obsahuje tie isté dáta ako verejný, a navyše k nim dáta potrebné na odšifrovanie. Tento systém dovoľuje bezpečné zašifrovanie správy takým spôsobom, aby ju dokázala dešifrovať len jediná entita.

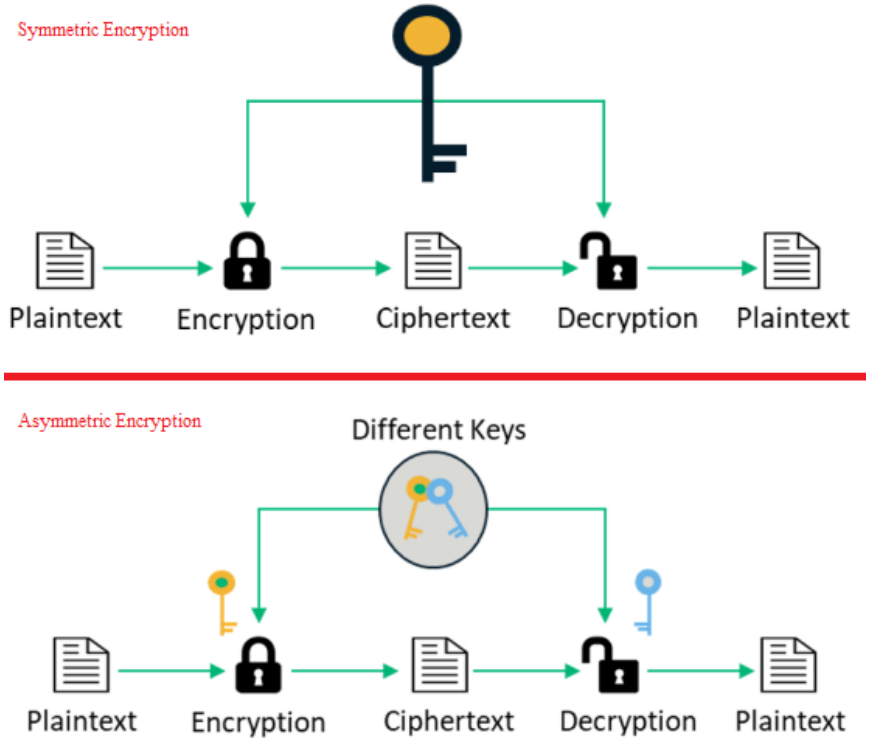
Vďaka existencii verejného kľúča na rozdiel od symetrickej kryptografie dokážeme šifrovanie dosiahnuť bez počiatočnej výmeny kľúčov. To predstavuje obrovskú výhodu oproti symetrickému šifrovaniu, kde najväčší problém predstavuje spôsob výmeny kľúčov pred začatím komunikácie.



Obr. 1. Na obrázku chce John poslať správu Mary bez toho, aby dokázal útočník správu odchytiť a prečítať. Predtým s Mary nemal príležitosť vymeniť si kľúče potrebné na symetrické šifrovanie, a musí sa teda obrátiť na asymetrickú infraštruktúru. Zoberie Maryn verejný kľúč a zašifruje pomocou neho svoju správu. Zaujímavým dôsledkom fungovania asymetrickej infraštruktúry je, že aj keď správu zašifroval, nedokáže ju odšifrovať. Nakoniec zašifrovanú správu pošle Mary.

Po prijatí správy Mary použije svoj súkromný kľúč na odšifrovanie tejto správy. Ak bude chcieť Mary odpovedať Johnovi na správu a zachovať diskrétnosť ich rozhovoru, použije na zašifrovanie Johnov verejný kľúč.

Nevýhodu oproti symetrickému šifrovaniu je pomalšia rýchlosť šifrovania, keďže asymetrické šifrovanie využíva zložitejšie algoritmy oproti symetrickému, ktoré väčšinou používa prevažne bitové a maticové operácie, ktoré sú pre počítače veľmi efektívne optimalizované.



Obr. 2. Na obrázku je vidieť rozdiel medzi symetrickým a asymetrickým šifrovaním, presnejšie to, že symetrické šifrovanie používa jediný kľúč na zašifrovanie aj odšifrovanie, zatiaľ čo asymetrické používa rozdielne kľúče.

Kvôli tomu sa často používa hybridné šifrovanie, pri ktorom sa symetrické kľúče zašifrujú pomocou asymetrického šifrovania, a po výmene asymetricky zašifrovaných symetrických kľúčov zariadenia komunikujú cez symetrické šifrovanie.

Tým sa zbavujeme najväčšieho problému symetrického šifrovania, ktorým je bezpečná výmena kľúčov pred začatím komunikácie, a zároveň sa zbavujeme nevýhody asymetrického šifrovania, ktorou je pomalšie šifrovania.

1.2 Digitálne pečiatky

Asymetrické šifrovanie sa tiež používa na tvorbu digitálnych pečiatok. Pri digitálnych pečiatkach sa role kľúčov vymenia. Súkromný kľúč slúži na vytvorenie danej pečiatky, zatiaľ čo verejný kľúč slúži na overenie toho, či pečiatka naozaj bola vytvorená

danou entitou.

Ak chce entita aplikovať digitálnu pečiatku na skupinu dát a dať tým verejne najavo, že dáta boli podpísané touto entitou, vygeneruje pomocou hash funkcie hodnotu, kde ako vstup sú podpisované dáta (často sa ako hash funkcia na tento účel používa SHA 256) a na hash hodnotu aplikuje svoj verejný kľúč. Tým tento hash podpíše a vytvorí novú hodnotu, ktorú priloží ku dátam ako svoj digitálny podpis.

Na podpisovanie sa používajú hash hodnoty a nie samotné dáta, lebo hashovacia funkcia vráti kratšiu hodnotu ako sú dáta, a preto podpisovanie zaberie menej času. Vďaka vlastnostiam kryptografických hash funkcií nie je z hľadiska bezpečnosti rozdiel medzi podpisovaním dát a podpisovaním ich hash hodnoty.

Vlastnosti, ktoré to zaručujú, sú nasledovné:

1. Pre akýkoľvek výstup kryptografickej hash funkcie nie je vypočítateľne možné nájsť vstup, ktorý by vrátil danú hodnotu
2. Nie je vypočítateľne možné nájsť dve hodnoty, ktoré vrátia tú istú hash hodnotu.
3. Nie je vypočítateľne možné nájsť vstup pre kryptografickú hash funkciu, ktorá by vrátila určitú hash hodnotu

Vďaka týmto vlastnostiam je zaručené, že aj keď je podpísaná jedine hash hodnota a nie kompletné dáta, útočník to nedokáže zneužiť.

Povedzme, že by chcel využiť fakt, že rôzne vstupy môžu vrátiť tú istú hash hodnotu. Útočník by sa mohol pre daný podpis pokúsiť nájsť inú hodnotu, pre ktorú by podpis bol platný. Niečo také ale nie je z praktického hľadiska možné, lebo nájdenie takejto hodnoty by bolo v rozpore s druhou a treťou vlastnosťou. Z tohto dôvodu je zneužitie digitálneho podpisu hash hodnoty vypočítateľne nemožné.

Pri podpisovaní hash hodnôt využívame ešte jednu vlastnosť kryptografických hash funkcií:

4. Vypočítanie hash hodnoty je rýchle pre akýkoľvek vstup.

Vďaka tomu dokážeme podpísať dáta v akomkoľvek rozsahu s konštantou časovou zložitosťou.

1.3 RSA

Najčastejšou funkciou pre asymetrickú kryptografiu je RSA. Je založená na modulo operácii a využíva viaceré matematické pravidlá pre dosiahnutie bezpečnosti. Funguje nasledovne:

(pre jednoduchosť pri modulo operáciach namiesto znamienka ≡ budeme používať =)

Najprv vygenerujeme dve prvočísla p a q. Z týchto prvočísiel vypočítame číslo n tak, že:

$n = p * q$.

Potom si vyberieme číslo e tak, aby e ∈ $Z^*_{\varphi(\mathrm{n})}$ kde:

$\varphi(\mathrm{n})$ = eulerova funkcia, ktorá vráti počet čísel menších ako n, ktoré sú nesúdeliteľné s n.

$Z^*_{\varphi(\mathrm{n})}$ = množina všetkých čísel menších ako $\varphi(\mathrm{n})$, ktoré sú nesúdeliteľné s $\varphi(\mathrm{n})$.

Následne vypočítame multiplikatívny inverzný prvok pre e v module $\varphi(\mathrm{n})$:

$d = e^{-1} \bmod \varphi(n)$.

Súkromný kľúč bude nasledujúca trojica čísel = (n, e, d).

Verejný kľúč bude nasledujúca dvojica čísel = (n, e).

Ak máme správu m, ktorú chceme zašifrovať pomocou verejného kľúča, tak verejný kľúč na m aplikujeme:

$m^e \bmod n = c$,

kde c je zašifrovaná správa. Ak chceme správu odšifrovať, na zašifrovanú správu c aplikujeme súkromný kľúč:

$c^d \bmod n = m$.

Odšifrovanie funguje vďaka niekoľkým matematickým pravidlám. Na vysvetlenie princípu dešifrovania a vizualizáciu aplikovania jednotlivých matematických pravidiel, ktoré dešifrovanie umožňujú, si musíme do výrazu pre dešifrovanie dosadiť výraz m^e. Dešifrovanie teda vyjadríme spôsobom:

$c^d \bmod n = (m^e)^d \bmod n = m^{e * d} \bmod n$

Vďaka tomu, že d je multiplikatívnym inverzným prvkom e v mod n*, tak:*

$e * d \bmod \varphi(n) = 1$ (toto vyplýva z definície multiplikatívneho inverzného prvku).

Z pravidla $e * d \bmod \varphi(n) = 1$ *vyplýva, že* $e * d = \varphi(n) * a + 1$

To znamená, že $m^{e * d} \bmod n = m^{\varphi(n) * a + 1} \bmod n$

Teraz sa vrátime ku vzorcu eulerovej funkcie: $\varphi(\mathrm{n}) = n * \prod_{i=1}^{k}(1 - \frac{1}{p_i})$, kde n = $\sum_i^k p_i^{e_i}$

V našom prípade sa $n = p * q$, čo znamená, že $\varphi(\mathrm{n}) = p * q * \left(1 - \frac{1}{p}\right) * \left(1 - \frac{1}{q}\right)$.

Tento výraz dokážeme zjednodušiť nasledovne:

$$\varphi(\mathrm{n}) = p * q * \left(1 - \frac{1}{p}\right) * \left(1 - \frac{1}{q}\right) = \left(p * \left(1 - \frac{1}{p}\right)\right)\left(q * \left(1 - \frac{1}{q}\right)\right) = (p-1) * (q-1)$$

Teraz musíme použiť matematické pravidlo fermatovej malej vety, ktoré hovorí:

Ak p je prvočíslo, tak pre akékoľvek a ∈ Z také, že p ∤ a platí, že a^{p-1} *mod p = 1.*

Ak sa vrátime k nášmu medzivýsledku $m^{\varphi(n) * a + 1}$ *mod n* a použijeme fakt, že:

$\varphi(n) = (p-1) * (q-1)$,

tak:

$m^{(p-1) * (q-1) * a + 1}$ *mod n.*

Ak by sme výraz trocha upravili a dostali výraz $m^{(p-1) * (q-1) * a}\ mod\ p$, tak vyplýva že:

$m^{(p-1) * (q-1) * a}\ mod\ p = m^{(p-1) * k}\ mod\ p$, kde *k = (q - 1) * a*.

Podľa fermatovej malej vety vyplýva že m^{p-1} *mod p = 1*, a teda aj

$m^{(p-1) * k}\ mod\ p = 1$,

pretože:

$m^{(p-1)} \bmod \mathrm{p} = 1$

čo znamená, že:

$\prod_{i=1}^{k} m^{p-1}\ mod\ p = \prod_{i=1}^{k} 1\ mod\ p = 1.$

To isté platí pre mod q, teda:

$m^{(p-1) * (q-1) * a}\ mod\ q = 1.$

Takže máme dve rovnice:

$m^{(p-1) * (q-1) * a}\ mod\ p = 1$

$m^{(p-1) * (q-1) * a}\ mod\ q = 1$

Podľa čínskej vety o zvyškoch vyplýva, že ak:

x mod p = a,

x mod q = a,

p a q sú nesúdeliteľné, tak:

*x mod p * q = a.*

Túto vetu aplikujeme na našu sústavu rovníc. Keďže:

$m^{(p-1)*(q-1)*a} \; mod \; p = 1$

$m^{(p-1)*(q-1)*a} \; mod \; q = 1$

tak aj

$m^{(p-1)*(q-1)*a} \; mod \; p * \; q = 1.$

Ak sa vrátime k nášmu počiatočnému príkladu a aplikujeme tieto pravidlá v poradí, tak dostaneme nasledujúce kroky:

$$c^d \; mod \; n = (m^e)^d \; mod \; n = m^{e*d} \; mod \; n = m^{\varphi(\mathrm{n})*\mathrm{a}+1} \; mod \; n = m^{\varphi(\mathrm{n})*\mathrm{a}} \; * m \; mod \; n$$
$$= m^{(p-1)*(q-1)*a} * m \; mod \; n = 1 * m \; mod \; n = m.$$

Teda $c^d \; mod \; n = m$

Asymetrické šifrovanie RSA je bezpečné, pretože aj keď útočník pozná verejný kľúč (e, n), nedokáže realisticky získať chýbajúcu časť d nachádzajúcu sa v súkromnom kľúči.

Je to spôsobené tým, že na vypočítanie d potrebuje použiť rozšírený euklidov algoritmus pri module $\varphi(\mathrm{n})$. Na výpočet $\varphi(\mathrm{n})$ potrebuje poznať podľa vzorca euklidovej funkcie všetky prvočísla, z ktorého sa dané číslo skladá.

Tu nastáva pre útočníka kľúčový problém, na ktorom je založená bezpečnosť RSA: Vypočítať defaktorizáciu dostatočne veľkého čísla n je s modernými technológiami a matematickými poznatkami výpočtovo nezvládnuteľné. Preto útočník nikdy nezíska prístup ku číslam p a q, a teda nevypočíta ani $\varphi(\mathrm{n})$ a d.

Veľkou nevýhodou RSA šifrovania je obmedzené množstvo dát, ktoré ním môže byť zašifrované naraz. Toto množstvo je ekvivalentné veľkosti kľúča mínus veľkosti paddingu.

1.4 Kvantové počítanie

Prelomiť šifru RSA je z teoretického hľadiska možné pomocou dosiaľ neexistujúcich kvantových superpočítačov. Najvýkonnejší funkčný kvantový počítač dnes dokáže využiť 53 kubitov, zatiaľ čo predpokladaný počet kubitov potrebných na prelomenie RSA je odhadovaný na 1500.

Kryptografické organizácie aj napriek tomu prichádzajú s novými štandardmi a metódami, ktoré prinesú robustnejšie kvantovo neprelomiteľné spôsoby asymetrického šifrovania, ktoré RSA nahradia.

1.5 Digitálne certifikáty

Komunikácia na internete s neznámymi entitami môže predstavovať bezpečnostnú hrozbu. Pred navštívením webovej stránky a nadviazaním kontaktu by sme chceli ideálne overiť, či táto stránka nie je nebezpečná. Na to slúžia digitálne certifikáty.

Tieto certifikáty sú vydávané dôvernými webovými entitami, ktoré vydaním certifikátu pre určitú stránku zabezpečujú pre návštevníkov tejto stránky, že je legitímna. Entity vydávajúce tieto certifikáty sa nazývajú certifikačné autority.

Vďaka systému digitálnych certifikátov zariadenie začne komunikáciu s webovou stránkou jedine v prípade, že táto stránka obsahuje certifikát vydaný valídnou certifikačnou autoritou. Certifikáty využívajú pri ich generácií a overovaní infraštruktúru verejného kľúča. Rozprávame sa teda o prostredí, v ktorom každá entita má svoj verejný a súkromný kľúč.

O obsahu certifikátu rozhoduje medzinárodný štandard X.509. Podľa tohto štandardu každý certifikát obsahuje nasledujúce informácie:

1. **Verzia:**
   Určuje verziu certifikátu (napr. X.509 v1, v2, alebo v3). Najnovšia a najpoužívanejšia je verzia 3 (v3), ktorá umožňuje používanie rozšírení (extensions).
2. **Sériové číslo:**
   Unikátne číslo pridelené certifikátu certifikačnou autoritou (CA).
3. **Algoritmus podpisu:**
   Algoritmus použitý na digitálny podpis certifikátu, napr. RSA alebo ECDSA spolu s hashovacím algoritmom ako SHA256.
4. **Informácie o vydavateľovi:**
   Informácie o certifikačnej autorite, ktorá certifikát vydala (napr. názov, krajina).
5. **Platnosť certifikátu:**
   Obsahuje:
   - Dátum začiatku platnosti.
   - Dátum expirácie.

6. **Predmet certifikátu (Subject):**
   Identifikuje entitu, ktorej certifikát patrí (napr. názov domény, organizácie, e-mailová adresa).
7. **Verejný kľúč:**
   Verejný kľúč, ktorý je súčasťou páru verejný/súkromný kľúč.
8. **Rozšírenia certifikátu (Extensions):**
   Pridané v X.509 v3. Rozšírenia umožňujú definovať dodatočné atribúty, napr.:
   - Key Usage: Špecifikuje účel kľúča (napr. šifrovanie, podpisovanie).
   - Subject Alternative Name (SAN): Umožňuje zahrnúť alternatívne názvy (napr. viac domén v jednom certifikáte).
   - Certificate Policies: Informácie o pravidlách CA.
9. **Digitálny podpis certifikátu:**
   Podpis certifikátu vytvorený súkromným kľúčom CA, ktorý overuje pravosť a integritu certifikátu.

Aby webová stránka získala certifikát, potrebuje o neho požiadať od certifikačnej autority. Takáto žiadosť sa nazýva CSR (Certificate Signing Request). Organizácia požiada o certifikát v prípade, že chcú vytvoriť bezpečnú webovú stránku vytvorenú pre online návštevníkov, napríklad stránku pre online obchod.

CSR sa skladá sa z nasledujúcich predmetov:

- Verejný kľúč žiadateľa.
- Informácie o žiadateľovi (názov domény, organizácie, krajina atď.).
- Algoritmus, ktorý sa použije na vytvorenie certifikátu (napr. RSA).

CSR sa odošle certifikačnej autorite. Tá musí následne overiť, či žiadateľ má právo používať danú doménu alebo zastupovať organizáciu. Spôsob overenia závisí od typu certifikátu:

Domain Validation (DV) je základný typ certifikácie, pri ktorom CA overuje, či žiadateľ má kontrolu nad doménou, pre ktorú požaduje certifikát. Tento proces overenia je jednoduchý a rýchly, pretože sa sústredí výhradne na vlastníctvo domény, bez overovania identity organizácie alebo jednotlivca.

Certifikačná autorita môže na overenie použiť jednu z nasledujúcich metód:

1. Odoslanie špecifického súboru na server žiadateľa. CA poskytne presný obsah súboru, ktorý musí byť nahraný na presne určené miesto na serveri, aby bolo možné potvrdiť vlastníctvo domény.
2. Vytvorenie špecifického DNS záznamu. Žiadateľ musí do DNS konfigurácie svojej domény pridať konkrétny textový (TXT) alebo CNAME záznam poskytnutý CA, ktorý autorita následne overí.

3. Odoslanie overovacieho e-mailu na administratívnu e-mailovú adresu spojenú s doménou. Bežne používané adresy sú napríklad admin@domain.com, webmaster@domain.com alebo hostmaster@domain.com. Žiadateľ musí odpovedať na tento e-mail, aby preukázal svoju kontrolu nad doménou.

Organization Validation (OV) poskytuje vyššiu úroveň dôveryhodnosti v porovnaní s DV certifikátmi. Okrem overenia vlastníctva domény certifikačná autorita tiež overuje existenciu a identitu organizácie, ktorá o certifikát žiada. Tento proces zahŕňa:

- Poskytnutie oficiálnych dokumentov, ktoré potvrdzujú právny status organizácie. Medzi tieto dokumenty môžu patriť výpis z obchodného registra, certifikáty o registrácii alebo licencie na podnikanie.
- Overenie fyzickej adresy a kontaktných informácií organizácie. Certifikačná autorita môže napríklad kontaktovať organizáciu prostredníctvom telefonického hovoru na verejne dostupné telefónne číslo.
- Overenie právneho názvu organizácie, ktorý bude uvedený v certifikáte.

Výsledkom je certifikát, ktorý poskytuje dôveru, že webová stránka patrí konkrétnej organizácii a táto organizácia bola overená nezávislou autoritou.

Extended Validation (EV) predstavuje najvyššiu úroveň overenia, ktorú môže certifikačná autorita poskytnúť. Certifikát s EV overením prináša maximálnu dôveryhodnosť, čo je obzvlášť dôležité pre webové stránky bánk, e-commerce spoločností alebo iné stránky, kde dochádza k citlivým transakciám. Proces overenia zahŕňa:

- Dôkladnú kontrolu právnej existencie organizácie. Certifikačná autorita preveruje organizáciu v oficiálnych registroch, aby potvrdila jej právny status a oprávnenie na podnikanie.
- Overenie fyzickej adresy organizácie. Certifikačná autorita preveruje, či organizácia má platnú a overiteľnú fyzickú adresu, na ktorej je možné ju kontaktovať.
- Overenie totožnosti a oprávnení osoby, ktorá podáva žiadosť o certifikát. CA môže vyžadovať identifikáciu osoby a potvrdenie jej právomocí konať v mene organizácie.
- Manuálnu kontrolu všetkých poskytnutých informácií zo strany CA, čo predlžuje proces vydania certifikátu, ale zároveň zvyšuje jeho dôveryhodnosť.

Certifikát s EV overením je ľahko rozpoznateľný, pretože prehliadače často zobrazujú názov organizácie v paneli adresy vedľa URL, čím zdôrazňujú dôveryhodnosť webovej stránky.

Po úspešnom overení CA vydá digitálny certifikát. Tento certifikát obsahuje:

- **Verejný kľúč žiadateľa.**
- **Informácie o entite (napr. názov domény alebo organizácie).**
- **Platnosť certifikátu (od-do).**

- **Digitálny podpis certifikačnej autority.**

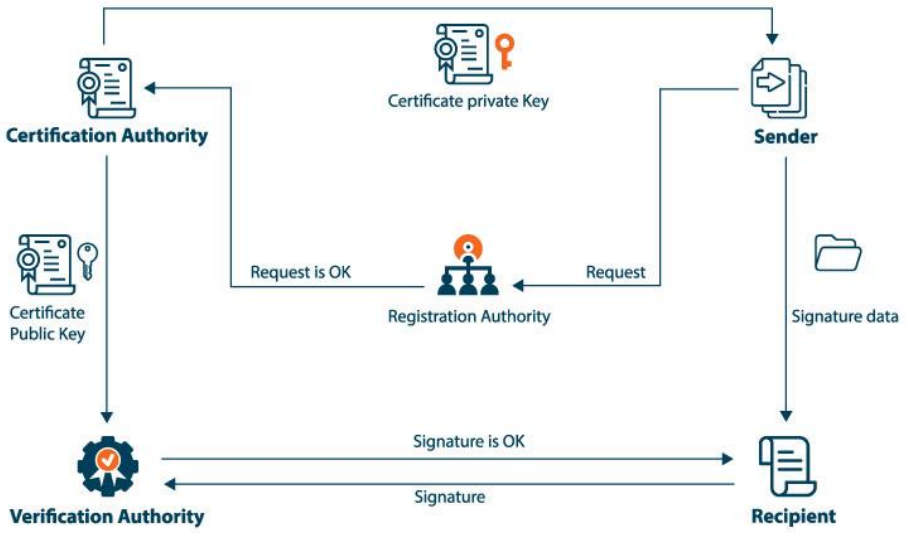
Certifikát je podpísaný pomocou súkromného kľúča CA. Vďaka vyššie vysvetlenému princípu asymetrickej kryptografie je podpisom pomocou súkromného kľúča zaručená pravosť dokumentu.

Po vydaní certifikátu ho žiadateľ nasadí na svoj server. Typickým príkladom je použitie certifikátu na webovom serveri, ktorý umožňuje zabezpečenú komunikáciu cez HTTPS.

Okrem vytvorenia a nasadenia certifikátu je potrebným krokom aj overenie jeho platnosti. Keď klient (napr. webový prehliadač) navštívi stránku, server mu poskytne certifikát. Prehliadač vykoná tieto kroky:

1. **Overenie digitálneho podpisu certifikátu:**
   Prehliadač skontroluje, či bol certifikát podpísaný dôveryhodnou CA. Dôveryhodné CA sú predinštalované v prehliadačoch a operačných systémoch (tzv. koreňové certifikáty).
2. **Kontrola platnosti certifikátu:**
   Overí sa, či je certifikát stále platný (dátum vydania a expirácie).
3. **Zabezpečenie šifrovanej komunikácie:**
   Prehliadač a server vytvoria šifrované spojenie pomocou asymetrickej a symetrickej kryptografie (napr. protokol TLS).

Entita, ktorá pomáha zariadeniam pri overovaní certifikátov (napríklad poskytnutím zoznamu vydaných certifikátov) sa nazýva verifikačná autorita.



Obr. 3. Obrázok znázorňuje interakciu certifikačnej autority so žiadateľom o certifikát.

Pre jasnejšie overovanie certifikátov vznikli takzvané CT logy (certificate transparency logs). Sú to verejné, kryptograficky zabezpečené denníky, ktoré slúžia na zaznamenávanie všetkých digitálnych certifikátov vydaných certifikačnými autoritami (CA).

Tento systém bol vytvorený na zvýšenie bezpečnosti a transparentnosti v infraštruktúre verejných kľúčov a na odhaľovanie neoprávneného vydávania certifikátov. Systém podporuje:

1. **Transparentnosť:**
   Všetky vydané certifikáty sú verejne dostupné, čo umožňuje ich audit a overovanie.
2. **Zníženie rizika zneužitia:**
   Ak CA vydá falošný alebo neoprávnený certifikát (napr. pre doménu, ktorú nevlastní príslušný žiadateľ), tento certifikát môže byť okamžite odhalený.
3. **Zvýšenú dôveru:**
   Majitelia domén a organizácie môžu sledovať certifikáty vydané pre svoje domény a identifikovať potenciálne hrozby.

Vydávanie certifikátov len malou plne dôveryhodnou skupinou entít je nerealistické kvôli veľkosti digitálneho priestoru. Preto majú CA možnosť udeľovať práva na udávanie certifikátov ďalším entitám, čím vznikajú nové certifikačné autority. Tieto nové autority sú uložené na nižšej vrstve vo vnútri CA hierarchie. Táto hierarchia funguje nasledovne:

- **Koreňová certifikačná autorita (Root CA):**
  Koreňové certifikáty sú najdôveryhodnejšie a sú predinštalované v systémoch. Tieto certifikáty podpisujú sprostredkovateľské CA.
- **Sprostredkovateľské certifikačné autority (Intermediate CA):**
  Tieto CA vydávajú certifikáty pre konečných užívateľov (napr. webové stránky). Použitie sprostredkovateľských CA umožňuje izolovať koreňové certifikáty a znížiť riziko ich kompromitácie.
- **Konečný certifikát (End-Entity Certificate):**
  Toto je certifikát vydaný žiadateľovi, ktorý je nasadený na serveri.

Prehliadač overuje celý reťazec dôvery od konečného certifikátu až po koreňový certifikát.

Ak sa zistí, že certifikát bol kompromitovaný alebo už nie je platný (napr. doména už nepatrí danej organizácii), CA môže certifikát zneplatniť:

- **CRL (Certificate Revocation List):** Zoznam zneplatnených certifikátov, ktorý je pravidelne aktualizovaný.
- **OCSP (Online Certificate Status Protocol):** Dynamický spôsob overovania platnosti certifikátu v reálnom čase.

Príklady certifikačných autorít:

- **Let's Encrypt:** Bezplatná CA poskytujúca automatizované DV certifikáty.
- **DigiCert, GlobalSign, Comodo:** Komerčné CA ponúkajúce certifikáty na rôzne účely.

Systém certifikátov a certifikačných autorít zabezpečuje bezpečnú komunikáciu na internete a umožňuje používateľom dôverovať entitám, s ktorými komunikujú.

2. Praktická časť

Praktická časť môjho projektu sa skladá z nasledujúcich krokov:

- Vytvorenie infraštruktúry verejného kľúča pomocou kryptografických knižníc
- Vytvorenie vlastnej certifikačnej autority
- Vytvorenie možnosti posielania žiadostí o certifikát CSR servermi v infraštruktúre
- Podpisovanie CSR certifikačnou autoritou
- Vytvorenie transparentných certifikačných logov
- Vytvorenie listu revokovaných certifikátov, ktoré boli vydané, ale bola im odobraná platnosť
- Simulácia jednoduchého webového serveru

- Spôsob pre klienta verifikovať platnosť certifikátu štandardnými spôsobmi

Na tvorbu projektu som použil programovací jazyk python. Na splnenie jednotlivých funkcionalít som použil python knižnice:

- **cryptography** na kryptografické funkcie
- **datetime** na zadávanie dátumu vydania certifikátu a kontrolu platnosti dátumov
- **flask** na simuláciu webového servera
- **pickle** na serializáciu a deserializáciu správ pre simuláciu posielania správ cez sieť

2.1. Triedy

Simulácia jednotlivých entít v infraštruktúre je dosiahnutá pomocou tried. Nachádzajú sa tu nasledujúce triedy:

- Server
- Client
- Certificate_Authority
- Web_Application

Trieda Server predstavuje entitu, ktorá chce poskytovať štandardné serverové služby pre klientov. Môže sa jednať o organizáciu poskytujúcu služby v podobe webového servera.

Dokáže certifikačnú autoritu požiadať o certifikát pomocou vytvorenia CSR. V prípade úspešného získania certifikátu dokáže uložiť tento certifikát do súboru s príponou .pem (čo predstavuje uloženie tohto certifikátu na simulovaný server).

Keďže server je súčasťou infraštruktúry verejného kľúča, tak obsahuje aj funkcie na tvorbu páru kľúčov, šifrovanie a dešifrovanie správ, ako aj tvorbu a posielanie správ iným entitám v infraštruktúre. Server má tiež možnosť spustiť webovú aplikáciu.

Server má aj funkciu na vytvorenie vlastného certifikátu, ktorý si sám podpíše. Táto funkcia slúži na demonštráciu toho, že takéto falošné certifikáty nevydané certifikačnou autoritou neprejdú úspešne kontrolou platnosti certifikátov.

Trieda Client predstavuje klienta, teda zariadenie, ktoré chce komunikovať so serverom v klient-server komunikácií. V reálnom svete by pravdepodobne išlo o jednotlivca snažiaceho sa využiť serverové služby.

Klient má rovnaké možnosti ako server týkajúce sa generácie páru kľúčov, šifrovania, dešifrovania, tvorby a posielania správ.

Má tiež možnosť skontrolovať platnosť certifikátu. Pri kontrolovaní platnosti skontroluje digitálny podpis certifikačnej autority, a potom pošle certifikát samotnej CA, ktorá hlbšie skontroluje platnosť podľa ďalších podmienok. Kontrola certifikátu sa používa pri návšteve webových stránok klientom.

Trieda Certificate_Authority (CA) predstavuje certifikačnú autoritu. Má funkcionalitu pre podpisovanie CSR, ktoré dostane od serveru. Ako súčasť infraštruktúry verejného kľúča tiež vlastní svoj vlastný verejný a súkromný kľúč, ktoré používa pri podpisovaní CSR. Uchováva tiež záznamy o vydaných certifikátoch, ktoré necháva verejne prístupne v infraštruktúre (certificate transparency logs).

Funguje aj ako verifikačná autorita, pretože dokáže kontrolovať certifikátu posielané klientami na potvrdenie.

Pri kontrole certifikátu skontroluje nasledujúce veci:

- Bol daný certifikát vydaný touto autoritou?
- Je dátum certifikátu je expirovaný?
- Je certifikát je revokovaný (nachádza sa na certificate revocation list)?

V prípade kompletnej platnosti certifikátu vráti správu o tom, že certifikát je platný.

Trieda Web_Application predstavuje webovú aplikáciu spustenú serverom. Trieda obsahuje funkciu main a musí byť spustená zvlášť od hlavného programu, aby mohla správne fungovať. Na simuláciu webovej stránky sa používa knižnica Flask, a tá vyžaduje, aby funkčnosť stránky bežala na hlavnom threade programu. Preto projekt obsahuje dva programy, jeden pre bežanie stránky, a druhý pre všetko ostatné.

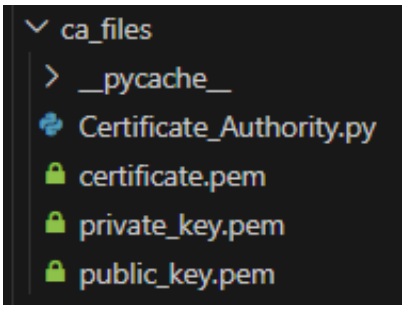
2.2. Reprezentácia kľúčov

Na správne fungovanie infraštruktúry musí každá entita vlastniť pár kľúčov. Pre triedu Client sú tieto kľúče uložené v statickej pamäti. Pri serveri a certifikačnej autorite sú kľúče uložené v podobe súboru s príponou .pem. Oba kľúče majú svoj vlastný súbor.

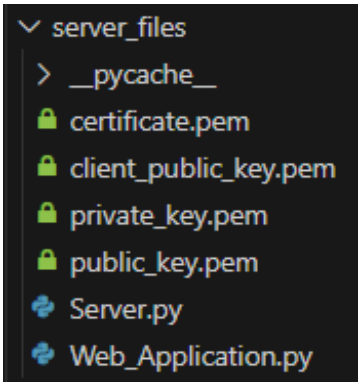
Okrem toho majú obe entity aj svoj vlastný certifikát. V prípade serveru je certifikát na začiatku behu programu prázdny súbor. Certifikačná autorita má uložený certifikát podpísaný samou sebou, ktorý vygeneruje na začiatku behu programu.

Server tiež obsahuje súbor client_public_key.pem pre asymetricky šifrovanú komunikáciu s klientom (na začiatku komunikácie si od klienta vypíta kľúč a zapíše ho do daného súboru).

Pem súbory sú uložené v dvoch priečinkoch, kde každý priečinok predstavuje prostriedky špecifickej entity. Súbory obsahujú aj súbor s kódom súvisiacim s danou entitou (pri certifikačnej autorite je to súbor Certificate_Authority.py a pri serveri sú to súbory Server.py a Web_Application.py).



Prostriedky certifikačnej autority



Prostriedky serveru

Klient nemá svoj vlastný priečinok, pretože kľúče ukladá v statickej pamäti. Súbor Client.py sa nachádza v priečinku spolu s main.py, ktorý riadi fungovanie

2.3. JSON requesty

Interakcia s webovou aplikáciou je veľmi jednoduchá. Stránka obsahuje jedinú informáciu, ktorou je premenná s textom. Klient dokáže interaktovať s týmto textom dvoma spôsobmi:

- GET
- POST

Metóda GET vráti aktuálny text uložený na stránke. Metóda POST text na stránke pozmení na novú špecifikovanú hodnotu.

Pred využitím týchto metód je potrebné nadviazať so serverom komunikáciu. To prebehne v podobe vzájomného preposlania verejných kľúčov. Klient si verejný kľúč servera uloží v statickej pamäti, server ho uloží v priečinku server_files pod menom client_public_key.pem.

Nadviazanie komunikácie nastane pripojením sa na doménu https://localhost:443/establish_connection. Metóda GET pre získanie textu sa posiela na https://localhost:443/get_text, a metóda POST na zmenu textu sa posiela na https://localhost:443/update_text. Samotná stránka https://localhost:443 zobrazí text vo webovom browseri.

Pri klient-server komunikácií si klient a server navzájom posielajú text vo formáte JSON. Po tom, čo klient požiada metódou GET o získanie textu zo stránky, server odošle JSON v nasledujúcom formáte:

```
json={"text": encrypted_text_b64}
```

Premenná encrypted_text_b64 obsahuje text zašifrovaný pomocou verejného kľúča servera. V JSON texte sa posiela táto správa pod kľúčom „text“.

Text sa posiela zo serveru na klient v prípade zavolania metódy GET. Vtedy server pošle v špecifikovanom JSON formáte zašifrovaný aktuálny text uložený na webovom serveri. V prípade metódy POST sa pošle zašifrovaný text z klienta na server v rovnakom formáte za účelom zmeny textu na serveri.

Okrem textu sa posielajú zo serveru na klient aj niektoré informačné správy vo formáte JSON. Informačné správy existujú nasledujúce:

```
{"message": "Public key received successfuly"}
```

Odošle sa v prípade, že poslanie verejného kľúča prebehne úspešne.

```
{"error": "No key provided"}
```

Odošle sa v prípade, že poslanie verejného kľúča neprebehne úspešne.

```
{"error": "No text provided"}
```

Odošle sa v prípade, že prijatý POST request neobsahuje text.

```
{"message": "Text updated successfully"}
```

Odošle sa v prípade, že prijatý POST request sa správne spracoval a text na stránke bol zmenený.

2.4. Konzolové príkazy

Na správne pracovanie programu je treba spustiť v konzole dva python súbory: main.py a Web_Application.py. Web_Application.py spustí pomocou knižnice Flask lokálnu simuláciu webového servera. Súbor main.py spustí fungovanie infraštruktúry a dovoľuje jej manipuláciu cez konzolové príkazy.

Main.py na začiatku inicializuje inštancie objektov Server, Client a Certificate_Authority. Z každého typu objektu sa inicializuje jedna inštancia: to znamená, že v infraštruktúre sa nachádza presne jeden klient, jeden server a jedna certifikačná autorita. Simulácia teda predstavuje minimálny rozsah prostredia, na ktorom je možné demonštrovať fungovanie digitálnych certifikátov.

Správanie určitej entity v infraštruktúre je možné napísaním mena danej entity a za ním pridaním výrazu „.príkaz“, kde príkaz bude hodnota samotného príkazu, ktorý má entita vykonať (Napríklad „client.msg“, kde „client“ je entita, a „msg“ je príkaz, ktorý má vykonať. Entita a príkaz sú oddelené bodkou.). Zoznam entít a ich možných príkazov je nasledovný:

Server

Možné príkazy:

1. **csr** => server vytvorí CSR a pošle ju CA, ktorá ju prijme a podpíše. Server podpísaný certifikát uloží v pem súbore.

```
if cmd == "csr":
    csr = server.create_csr("different_name.com")
    signed_cert = ca.receive_csr(csr)
    if signed_cert != None:
        server.save_cert(signed_cert)
    print("Successfully saved a valid certificate on the server.")
```

2. **fakecer** => server vytvorí falošný certifikát, ktorý uloží do pem súboru. Pre správne fungovanie programu je potrebné, aby sa najprv z databázy CA vymazal aktuálny certifikát serveru. Inak by server s validným certifikátom mohol tento certifikát prepísať pomocou príkazu “fakecer”, a potom by nedokázal požiadať o platný certifikát od CA, pretože CA by jeho starý certifikát malo stále uložený v aktuálnej databáze, a CA nemôže podpísať CSR, ktorý už podpísalo.

```
elif cmd == "fakecer":
    cert_pem = server.get_cert()
    ca.remove_certificate(cert_pem)
    server.save_self_signed_certificate()
    print("Successfully saved an invalid certificate on the server.")
```

Client

Možné príkazy:

1. **msg** => Po zadaní príkazu si program vypýta správu, ktorú chceme poslať na server. Po zadaní správy sa klient pokúsi verifikovať certifikát na serveri. Ak určí, že certifikát je neplatný, tak konzola vypíše kontrolnú správu o neplatnosti certifikátu. V opačnom prípade klient zašifruje správu a odošle ju zašifrovanú na server, ktorý po prijatí správu dešifruje a správa sa vypíše do konzoly.

```
if cmd == "msg":
    msg = input("Input what message you want to send to the server: ")

    if client.verify_certificate(server, ca):
        communication(server, client, msg)
    else:
        print("Can't communicate with the server because of an invalid certificate.")
```

```
def communication(server, client, msg):
    server_public_key_bytes = server.get_public_key()

    server_public_key = serialization.load_pem_public_key(server_public_key_bytes, backend=default_backend())
    encrypted_message = client.encrypt_message(server_public_key, msg)

    decrypted_message = server.decrypt_message(encrypted_message)
    print(f"Server successfully received the following message: {decrypted_message}")
```

2. **connect** => Pokúsi sa o pripojenie na webovú stránku. Najprv skontroluje platnosť certifikátu, podobne ako pri príkaze „msg“. Ak ide o platný certifikát, tak klient odošle svoj verejný kľúč serveru. Správa obsahujúca klientov verejný kľúč predstavuje pokus o vytvorenie relácie so serverom. V reálnom svete by tento proces bol zložitejší, ale v prostredí tohto projektu stačí serveru poslať JSON správu s verejným kľúčom, a server odpovie JSON správou potvrdzovacou spojenie.

```
def connect_to_app(self, server, ca):

    cert_is_valid = self.verify_certificate(server, ca)

    if cert_is_valid:
        print("Certificate check successful")

        self.send_public_key_to_web()

        print("Connection with website established")
        self.established_connection = True

    else:
        print("Can't connect to the website because of an invalid certificate")
```

3. **get** => Príkaz na získanie obsahu z webového servera. Najprv klient skontroluje, či bolo so serverom vytvorené spojenie. Následne odošle JSON správu na doménu s príponou

„/get_text“. V prípade úspešnej odpovede zo serveru sa daná odpoveď dešifruje a vypíše do konzoly. V prípade neúspešnej komunikácie sa do konzoly vypíše odpoveď zo serveru.

```
def request_text(self):

    if not self.established_connection:
        print("Client must first establish connection with the web page.")
        return

    get_response = requests.get(f"https://localhost:443/get_text", verify=False)

    if get_response.status_code == 200:
        current_text_encrypted = get_response.json().get("text", "")
        current_text_encrypted_bytes = base64.b64decode(current_text_encrypted)
        current_text = self.decrypt_message(current_text_encrypted_bytes)

        print("Text currently on the web page:", current_text)
    else:
        print("Failed to retrieve text:", get_response.json())
```

4. **post** => Príkaz na zmenu obsahu webového servera. Po zadaní príkazu si program vypýta správu, na ktorú chceme zmeniť obsah webovej stránky. Po zadaní správy klient skontroluje, či bolo so serverom vytvorené spojenie. Následne získa verejný kľúč servera, aby mohol správu zadanú v konzole zašifrovať. Zašifrovanú správu potom pošle vo forme JSON post request. Nakoniec podľa správy zo servera vypíše, či bolo zmenenie obsahu stránky úspešné.

```
def request_update(self, server, new_text):

    if not self.established_connection:
        print("Client must first establish connection with the web page.")
        return

    server_public_key_bytes = server.get_public_key()
    server_public_key = serialization.load_pem_public_key(server_public_key_bytes, backend=default_backend())
    encrypted_text = self.encrypt_message(server_public_key, new_text)
    encrypted_text_b64 = base64.b64encode(encrypted_text).decode('utf-8')

    update_response = requests.post(f"https://localhost:443/update_text", json={"text": encrypted_text_b64}, verify=False)

    if update_response.status_code == 200:
        print("Text updated successfully.")
    else:
        print("Failed to update text:", update_response.json())
```

CA

Možné príkazy:

1. **revoke** => Revokuje platnosť určitého certifikátu. To znamená, že ho uloží do revocation listu vo vnútri CA. Takýto certifikát je považovaný za vydaný ale neplatný.

2. **remove** => Vymaže daný certifikát z databázy vydaných certifikátov.

3. **log** => Vypíše logy uložené na CA.

2.5. Detaily šifrovania

Program používa na šifrovanie knižnicu cryptography. Na hashovanie sa používa funkcia SHA256. Na padding sa používa systém OAEP spolu s hash funkciou MGF1 na vytváranie pseudo náhodnej masky. Veľkosť kľúča je 2048 bitov a verejný exponent kľúča je 65537.

```
encrypted_message = public_key.encrypt(
    message.encode(),
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

Šifrovanie správy. Vidieť sú použité funkcie.

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
    backend=default_backend()
)
public_key = private_key.public_key()
```

Generácia páru kľúčov aj s parametrami

2.6. Zmeny od originálneho plánu

Od originálneho plánu na projekt, ktorý bol popísaný v spresnení témy sa zmenilo zopár vecí. Jedna z týchto vecí je, že namiesto chatovacej aplikácie, ktorá mala pôvodne fungovať na webovom serveri, funguje webový server ako databáza, z ktorej sa dajú odoberať a do ktorej sa dajú pridávať dáta. Táto zmena nastala kvôli tomu, že vytváranie chatovacej aplikácie by spôsobilo nadbytočné problémy, ktoré by nemali nič spoločné so samotnou kryptografiou alebo bezpečnosťou. Všetky bezpečnostné princípy, ktoré by sa na nej dali implementovať, sa dajú implementovať aj na jednoduchej databáze.

Tiež sa zmenil plán porovnávania rýchlosti asymetrickej RSA a symetrickej AES šifier, pretože som zistil, že RSA šifra sa používa len na prenos veľmi malých dát, kvôli jej veľkostnému obmedzeniu. Preto by som časovo porovnával len veľmi malé zlomky sekúnd, čo mi prišlo zbytočné.

3. Záver

Pri vytváraní mojej vlastnej infraštruktúry verejného kľúča som hlbšie pochopil fungovaniu generácie a spravovania párov kľúčov. Tiež som zistil viac o samotnej funkčnosti RSA šifry, ktorá sa často používa pri asymetrickej kryptografii. Získal som prehľad nad tým, s akými problémami sa môžeme stretnúť pri vytváraní bezpečných infraštruktúr.

Získal som aj znalosti o certifikačných autoritách a ako sa pomocou nich dosahuje bezpečnejší internet. Učil som sa o krokoch vo verifikácií certifikátov, ako aj kroky pri posielaní a podpisovaní žiadosti o certifikát. Vďaka práci na projekte teraz rozumiem roliam jednotlivých entít vo vnútri infraštruktúry a vzťahom medzi nimi.

3.1. Zdroje

Obr. 1.

https://www.google.com/search?client=opera&hs=9FO&sca_esv=bc90c0cb48f9edae&q=public+vs+private+key+infrastructure&udm=2&fbs=AEQNm0CC4-gMDzPNWtEma7IgWCuZ-m5WiASWc1HVC_Tgf6i9HRNfA9Szx7sMI9C3zSMpAVzmgKztDTbz77ypkLNTWLRLhuTgg-gq1kxkCrQ9T3fRDkbsBCWC6jBCPwBLxC9062KiGKj2BCr5epzvXvZODH5nJjB3cRdkEUDbNLQOxwbatJpZdAo&sa=X&ved=2ahUKEwjrwYXV_IyKAxV01QIHHSgyMeUQtKgLegQIFBAB&biw=1534&bih=704&dpr=1.25#vhid=pKh6n14AMUiwfM&vssid=mosaic

Obr. 2.

https://www.google.com/search?client=opera&hs=9FO&sca_esv=bc90c0cb48f9edae&q=public+vs+private+key+infrastructure&udm=2&fbs=AEQNm0CC4-gMDzPNWtEma7IgWCuZ-m5WiASWc1HVC_Tgf6i9HRNfA9Szx7sMI9C3zSMpAVzmgKztDTbz77ypkLNTWLRLhuTgg-gq1kxkCrQ9T3fRDkbsBCWC6jBCPwBLxC9062KiGKj2BCr5epzvXvZODH5nJjB3cRdkEUDbNLQOxwbatJpZdAo&sa=X&ved=2ahUKEwjrwYXV_IyKAxV01QIHHSgyMeUQtKgLegQIFBAB&biw=1534&bih=704&dpr=1.25#vhid=l3eq8IGs0tNv-M&vssid=mosaic

Obr. 3.

https://www.google.com/search?q=certificate+authority&client=opera&hs=eGO&sca_esv=bc90c0cb48f9edae&udm=2&biw=1534&bih=704&ei=6bFPZ--DE4Odi-gPpvOZmA4&ved=0ahUKEwivnNnj_IyKAxWDzgIHHaZ5BuMQ4dUDCBA&uact=5&oq=certificate+authority&gs_lp=EgNpbWciFWNlcnRpZmljYXRlIGF1dGhvcml0eTIFEAAYgAQyBRAAGIAEMgUQABiABDIFEAAYgAQyBRAAGIAEMgYQABgKGB4yBBAAGB4yBBAAGB4yBBAAGB4yBBAAGB5I7RlQAFiPFnAAeACQAQCYAXSgAcsPqgEEMTMuOLgBA8gBAPgBAZgCFaAC_w_CAgsQABiABBixAxiDAcICChAAGIAEGEMYigXCAggQABiABBixA8ICDhAAGIAEGLEDGIMBGIoFmAMAkgcEMTMuOKAHkG8&sclient=img#vhid=iks2pLqHOTI-WM&vssid=mosaic